

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 30, 2026 (1-2 hours)

Objective: Implement all 4 SSSP push variants.

Attempts made:

SSSP push follows the same structure as BFS but uses weighted distances. The main issue was the sentinel value `LLONG_MAX` needs a guard before adding weight to prevent overflow. I also initially compared `dist` directly against the Phase 2a file which uses -1 for unreachable vertices, but our internal representation uses `LLONG_MAX`, causing false mismatches.

Solutions/Workarounds:

Guard before adding weight:

```
if (prev[u] != LLONG_MAX) {
    long long candidate = prev[u] + w;
    if (dist[v] == LLONG_MAX || candidate < dist[v]) {
        dist[v] = candidate;
        updated[tid] = 1;
    }
}
```

Fixed Phase 2a comparison by using the `distance()` helper:

```
cpp
long long distance(uint32_t v) const {
    return (dist[v] == LLONG_MAX) ? -1 : dist[v];
}
```

Commits: Add SSSP serial naive push; Add SSSP serial dense worklist; Add SSSP parallel naive push with locks; Add SSSP parallel dense worklist with locks; Verify all 4 SSSP variants match each other and Phase 2a output

Experiments/Analysis:

Tested on the weighted graph (0 1 5 / 0 2 1 / 2 1 1 / 1 3 1 / 2 3 10). All 4 variants matched.

SSSP dense serial vs naive serial passed
SSSP naive parallel vs naive serial passed
SSSP dense parallel vs naive serial passed
SSSP naive serial vs Phase 2a passed

Learning outcome:

SSSP is the most sensitive to the sentinel value choice. Using `LLONG_MAX` directly without the overflow guard would silently produce wrong distances for vertices near the maximum value.